

Pandas

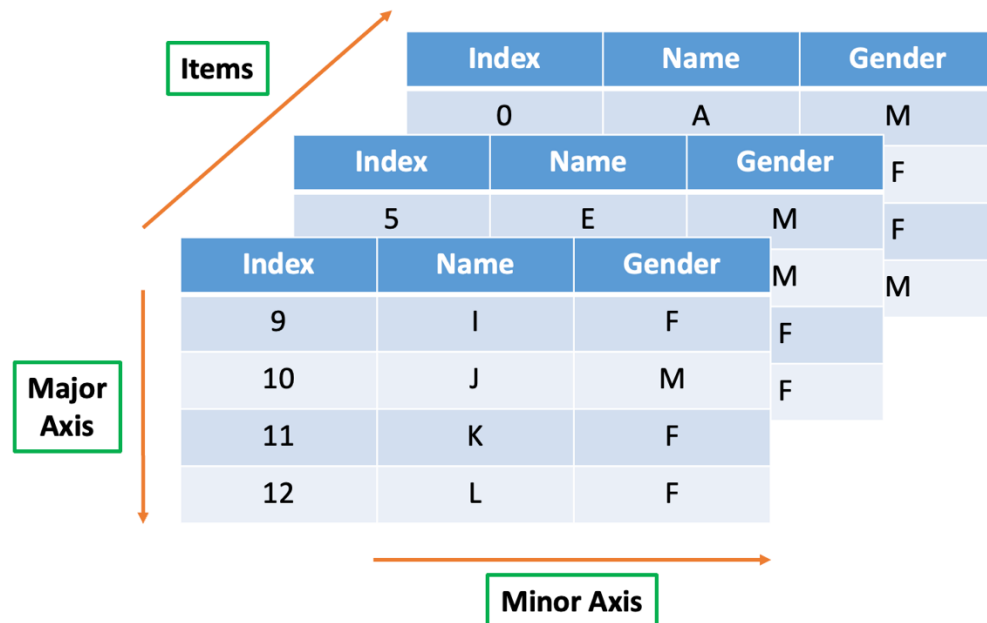
Extra



Dinh-Thang Duong – TA

Getting Started

❖ Objectives



The diagram illustrates the pandas data structure with a table and axes. An orange arrow labeled 'Items' points to the first row of the table. An orange arrow labeled 'Major Axis' points to the first column. An orange arrow labeled 'Minor Axis' points to the first row of the table.

Index	Name	Gender
0	A	M
5	E	M
9	I	F
10	J	M
11	K	F
12	L	F

Our objectives:

- Introduction to Pandas.
- Learn some essential operations in Pandas to fully manipulate .csv dataset.
- There are two problems corresponding to two specific data: regular tabular data and time-series data.

Outline

- Introduction
- Problem A
- Problem B
- Question

Introduction

Introduction

❖ Getting Started

Date	Consumption	Wind	Solar	Wind+Solar	Year	Month	Weekday
date	f64	f64	f64	f64	i32	u32	u32
2012-05-08	1354.063	84.137	143.577	227.714	2012	5	2
2010-02-11	1587.07	174.663	null	null	2010	2	4
2007-03-28	1442.523	null	null	null	2007	3	3
2009-06-08	1246.529	null	null	null	2009	6	1
2007-03-20	1486.952	null	null	null	2007	3	2

According we have a dataset like this

Introduction

❖ Getting Started

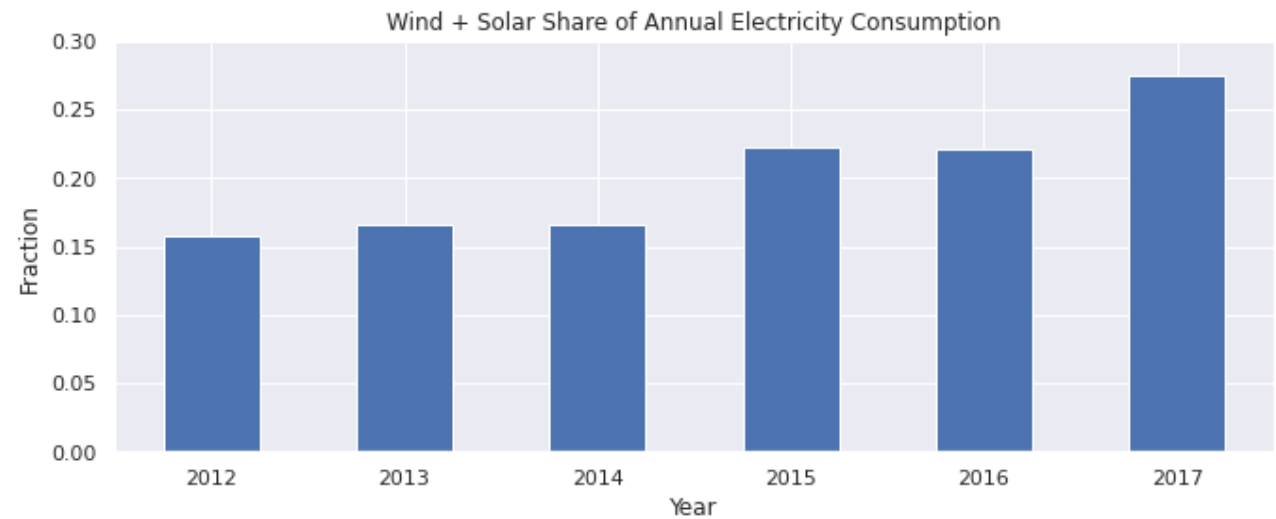
Date	Consumption	Wind	Solar	Wind+Solar	Year	Month	Weekday
date	f64	f64	f64	f64	i32	u32	u32
2012-05-08	1354.063	84.137	143.577	227.714	2012	5	2
2010-02-11	1587.07	174.663	null	null	2010	2	4
2007-03-28	1442.523	null	null	null	2007	3	3
2009-06-08	1246.529	null	null	null	2009	6	1
2007-03-20	1486.952	null	null	null	2007	3	2



	Consumption	Wind	Solar	Wind+Solar	Wind+Solar/Consumption
Year					
2015	505264.56300	77468.994	34907.138	112376.132	0.222410
2016	505927.35400	77008.126	34562.824	111570.950	0.220528
2017	504736.36939	102667.365	35882.643	138550.008	0.274500

Applying some operations

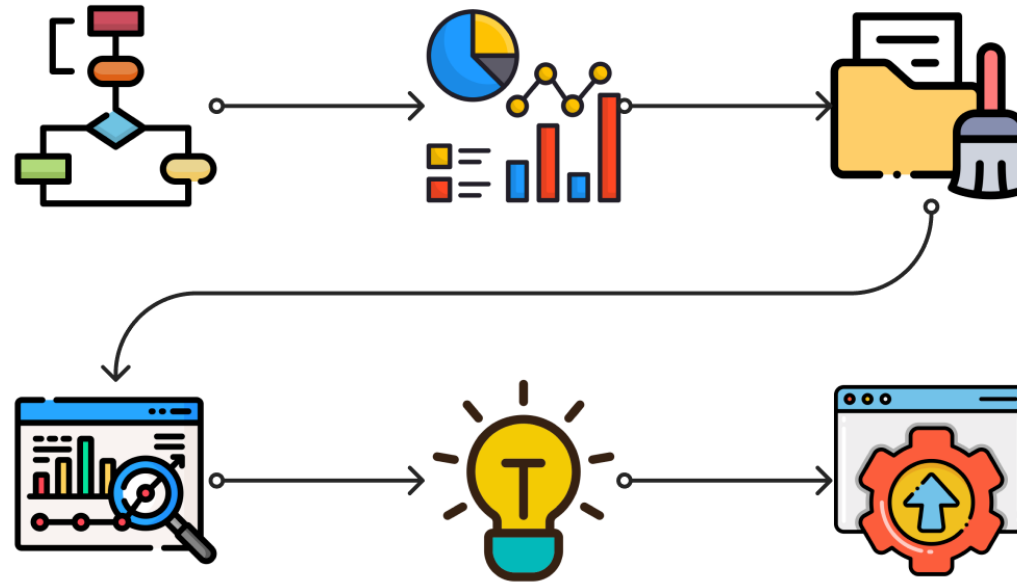
Visualization



Obsevation: The uptrending of using renewable energy

Introduction

❖ Getting Started



Data Analysis: A process of inspecting, cleansing, transforming and modeling data with the goal of discovering useful information, informing conclusions, and supporting decision-making.

Introduction

❖ Getting Started

Input table: 100,000,000 rows x 7 columns (5 GB)

Polars	0.8.8	2021-06-30	43s	
data.table	1.14.1	2021-06-30	92s	
ClickHouse	21.3.2.5	2021-05-12	159s	
spark	3.1.2	2021-05-31	332s	
DataFrames.jl	1.1.1	2021-06-03	349s	
dplyr	1.0.7	2021-06-20	370s	
(py)datatable	1.0.0a0	2021-06-30	500s	
pandas	1.2.5	2021-06-30	628s	
DuckDB	0.2.7	2021-06-15	630s	
dask	2021.04.1	2021-05-09	internal error	
cuDF*	0.19.2	2021-05-31	internal error	
Arrow	4.0.1	2021-05-31	not yet implemented	First time
Modin		see README	pending	Second time

Seconds 20 40 60 80 100 120

Data Analysis tools

Introduction

❖ Getting Started



Pandas: A Python library for data manipulation and analysis.

Introduction

❖ Getting Started

A Python library

Exploring

Manipulating

Cleaning

Analyzing

	A	B	C	D	E	F	G	H	I	J	K	L	M
1	#	Name	Type 1	Type 2	Total	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed	Generation	Legendary
2	1	Bulbasaur	Grass	Poison	318	45	49	49	65	65	45	1	FALSE
3	2	Ivysaur	Grass	Poison	405	60	62	63	80	80	60	1	FALSE
4	3	Venusaur	Grass	Poison	525	80	82	83	100	100	80	1	FALSE
5	3	VenusaurMe	Grass	Poison	625	80	100	123	122	120	80	1	FALSE
6	4	Charmander	Fire		309	39	52	43	60	50	65	1	FALSE
7	5	Charmeleon	Fire		405	58	64	58	80	65	80	1	FALSE
8	6	Charizard	Fire	Flying	534	78	84	78	109	85	100	1	FALSE
9	6	CharizardMe	Fire	Dragon	634	78	130	111	130	85	100	1	FALSE
10	6	CharizardMe	Fire	Flying	634	78	104	78	159	115	100	1	FALSE
11	7	Squirtle	Water		314	44	48	65	50	64	43	1	FALSE
12	8	Wartortle	Water		405	59	63	80	65	80	58	1	FALSE
13	9	Blastoise	Water		530	79	83	100	85	105	78	1	FALSE
14	9	BlastoiseMeg	Water		630	79	103	120	135	115	78	1	FALSE
15	10	Caterpie	Bug		195	45	30	35	20	20	45	1	FALSE
16	11	Metapod	Bug		205	50	20	55	25	25	30	1	FALSE
17	12	Butterfree	Bug	Flying	395	60	45	50	90	80	70	1	FALSE
18	13	Weedle	Bug	Poison	195	40	35	30	20	20	50	1	FALSE
19	14	Kakuna	Bug	Poison	205	45	25	50	25	25	35	1	FALSE
20	15	Beedrill	Bug	Poison	395	65	90	40	45	80	75	1	FALSE
21	15	BeedrillMega	Bug	Poison	495	65	150	40	15	80	145	1	FALSE
22	16	Pidgey	Normal	Flying	251	40	45	40	35	35	56	1	FALSE
23	17	Pidgeotto	Normal	Flying	349	63	60	55	50	50	71	1	FALSE
24	18	Pidgeot	Normal	Flying	479	83	80	75	70	70	101	1	FALSE
25	18	PidgeotMega	Normal	Flying	579	83	80	80	135	80	121	1	FALSE
26	19	Rattata	Normal		253	30	56	35	25	35	72	1	FALSE
27	20	Raticate	Normal		413	55	81	60	50	70	97	1	FALSE
28	21	Spearow	Normal	Flying	262	40	60	30	31	31	70	1	FALSE

Introduction

❖ Getting Started



Ease of Use

Visualization and Reporting

Learning Curve

Quick Prototyping

Data Entry and Formatting

Compatibility



Performance and Scalability

Automation and Reproducibility

Customization and Flexibility

Integration with Ecosystem

Version Control

Reproducibility and Portability

Introduction

❖ Pandas DataFrame

Columns

Rows

Index	Name	Gender
0	A	M
1	B	F
2	C	F
3	D	M

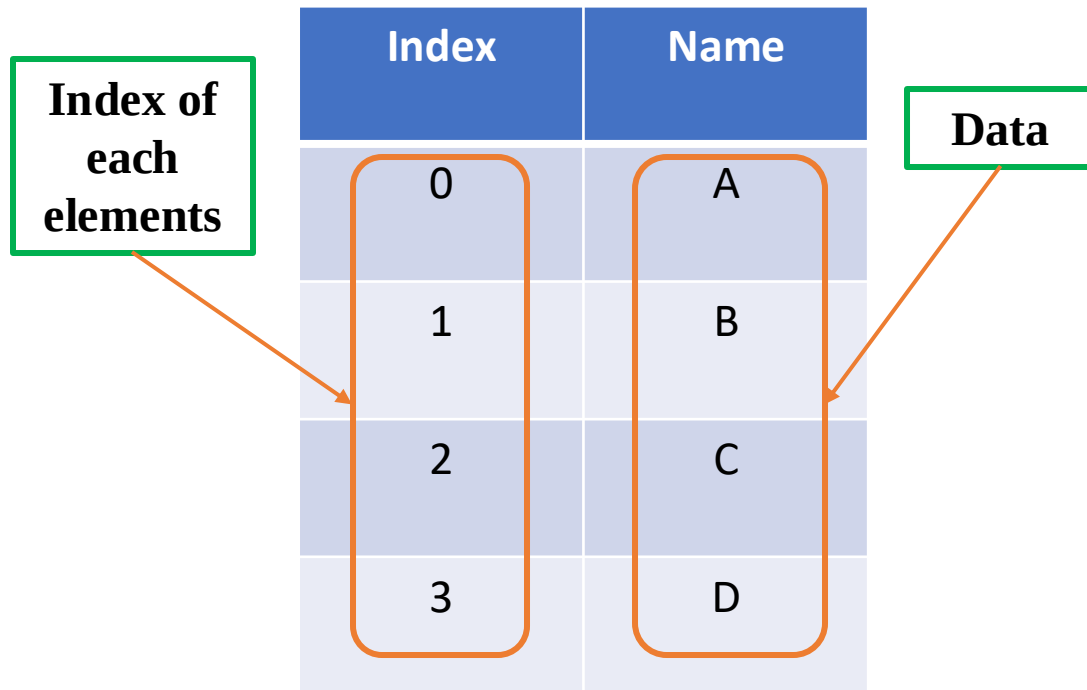
```
1 data = {  
2     'Name': ['A', 'B', 'C', 'D'],  
3     'Gender': ['M', 'F', 'F', 'M']  
4 }  
5  
6 df = pd.DataFrame(data)  
7 df
```

```
1 print(type(df))
```

```
<class 'pandas.core.frame.DataFrame'>
```

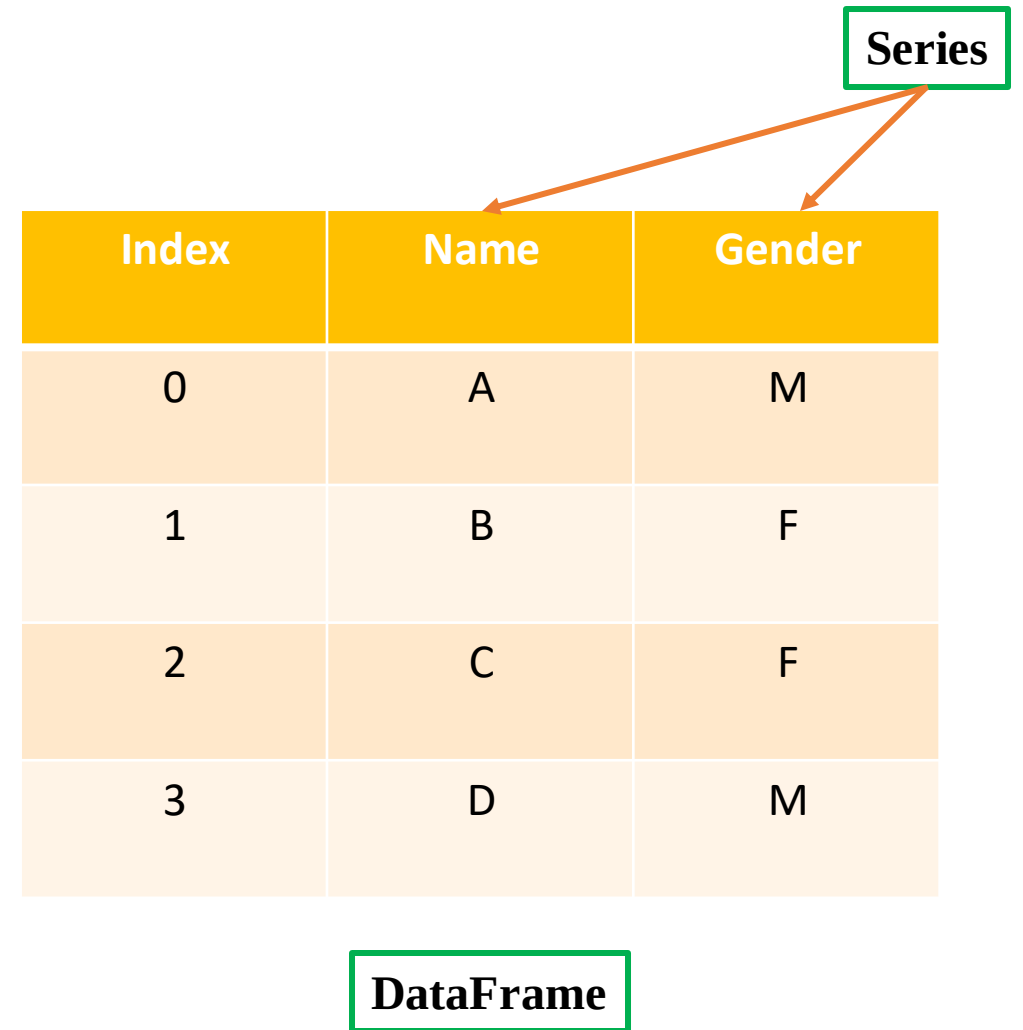
Introduction

❖ Pandas Series



The diagram illustrates a Pandas Series as a single column of data. It features a table with two columns: 'Index' and 'Name'. The 'Index' column contains values 0, 1, 2, and 3. The 'Name' column contains values A, B, C, and D. An orange rounded rectangle highlights the 'Index' column, with an arrow pointing to it from a green box labeled 'Index of each elements'. Another orange rounded rectangle highlights the 'Name' column, with an arrow pointing to it from a green box labeled 'Data'.

Index	Name
0	A
1	B
2	C
3	D



The diagram illustrates a Pandas DataFrame as a table with multiple columns. It features a table with three columns: 'Index', 'Name', and 'Gender'. The 'Index' column contains values 0, 1, 2, and 3. The 'Name' column contains values A, B, C, and D. The 'Gender' column contains values M, F, F, and M. An orange box labeled 'Series' has two arrows pointing to the 'Name' and 'Gender' columns. A green box labeled 'DataFrame' is positioned below the table.

Index	Name	Gender
0	A	M
1	B	F
2	C	F
3	D	M

Introduction

❖ Pandas Series

Index	Name
0	A
1	B
2	C
3	D

Series



Index	Gender
0	M
1	F
2	F
3	M

Series



Index	Name	Gender
0	A	M
1	B	F
2	C	F
3	D	M

DataFrame

```
1 name_series = df['Name']  
2 print(type(name_series))
```

```
<class 'pandas.core.series.Series'>
```

```
1 gender_series = df['Gender']  
2 print(type(gender_series))
```

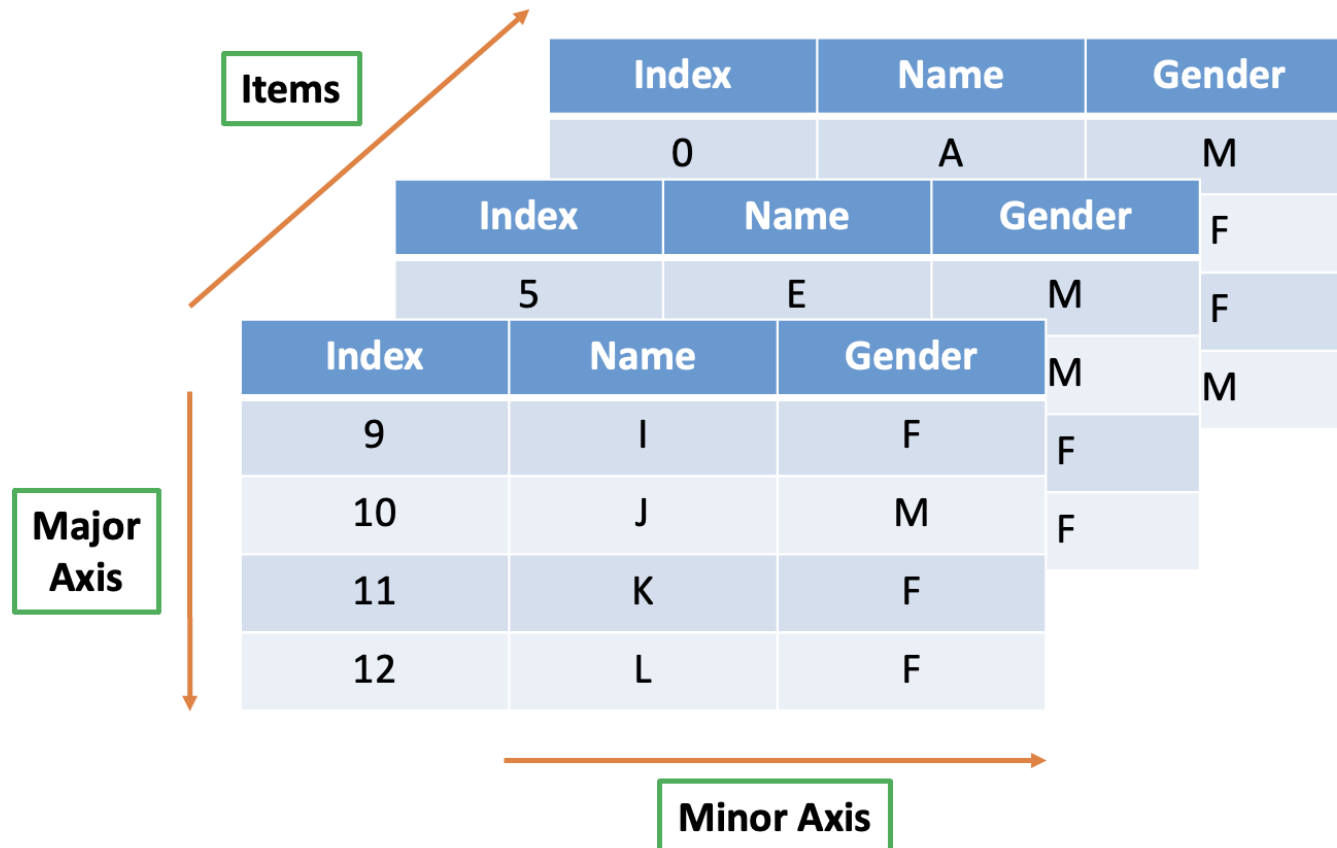
```
<class 'pandas.core.series.Series'>
```

```
1 new_df = pd.concat(  
2     [name_series, gender_series],  
3     axis=1  
4 )  
5 print(type(new_df))
```

```
<class 'pandas.core.frame.DataFrame'>
```

Introduction

❖ Pandas Panel



```
1 import xarray as xr
2
3 data_1 = {
4     'Name': ['A', 'B', 'C', 'D'],
5     'Gender': ['M', 'F', 'F', 'M']
6 }
7
8 data_2 = {
9     'Name': ['E', 'F', 'G', 'H', 'I'],
10    'Gender': ['M', 'M', 'F', 'F', 'M']
11 }
12
13 df1 = pd.DataFrame(data_1).to_xarray()
14 df2 = pd.DataFrame(data_2).to_xarray()
15
16
17 panel_data = xr.concat(
18     [df1, df2], dim='panel_dim'
19 )
20
21 panel_data
```

Problem A

Problem A

❖ Dataset

Description: Given [Pokemon.csv](#), use Pandas library to manipulate this dataset.

Pokemon with stats

2550New NotebookDownload (15 kB)

Data CardCode (1301)Discussion (22)Suggestions (0)

About Dataset

This data set includes 721 Pokemon, including their number, name, first and second type, and basic stats: HP, Attack, Defense, Special Attack, Special Defense, and Speed. It has been of great use when teaching statistics to kids. With certain types you can also give a geeky introduction to machine learning.

This are the raw attributes that are used for calculating how much damage an attack will do in the games. This dataset is about the pokemon games (*NOT* pokemon cards or Pokemon Go).

The data as described by [Myles O'Neill](#) is:

- **#:** ID for each pokemon
- **Name:** Name of each pokemon
- **Type 1:** Each pokemon has a type, this determines weakness/resistance to attacks
- **Type 2:** Some pokemon are dual type and have 2
- **Total:** sum of all stats that come after this, a general guide to how strong a pokemon is
- **HP:** hit points, or health, defines how much damage a pokemon can withstand before fainting
- **Attack:** the base modifier for normal attacks (eg. Scratch, Punch)
- **Defense:** the base damage resistance against normal attacks
- **SP Atk:** special attack, the base modifier for special attacks (e.g. fire blast, bubble beam)
- **SP Def:** the base damage resistance against special attacks
- **Speed:** determines which pokemon attacks first each round

Usability

8.82

License

[CC0: Public Domain](#)

Expected update frequency

Not specified

Tags

GamesVideo GamesAnime and MangaPopular Culture

Problem A

❖ Dataset

Description: Given [Pokemon.csv](#), use Pandas library to manipulate this dataset.

```
1 # https://drive.google.com/file/d/11fjFxxa2RfoLCJUQ\_OSTm3GfMZHVy8Gy/view?usp=sharing
2 !gdown --id 11fjFxxa2RfoLCJUQ_OSTm3GfMZHVy8Gy
```

Downloading...

From: <https://drive.google.com/uc?id=136oQPFJqG0vugwm0oA0U9IxxwJN0zT54K>

To: /content/Pokemon.csv

100% 44.0k/44.0k [00:00<00:00, 87.1MB/s]

Problem A

❖ Read CSV

```
1 import pandas as pd
2 import numpy as np
3
4 DATASET_PATH = '/content/Pokemon.csv'
5 data = pd.read_csv(DATASET_PATH)
```

```
import pandas as pd
pd.read_csv("data_path", **kwargs)
```

1 data

	#	Name	Type 1	Type 2	Total	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed	Generation	Legendary
0	1	Bulbasaur	Grass	Poison	318	45	49	49	65	65	45	1	False
1	2	Ivysaur	Grass	Poison	405	60	62	63	80	80	60	1	False
2	3	Venusaur	Grass	Poison	525	80	82	83	100	100	80	1	False
3	3	VenusaurMega Venusaur	Grass	Poison	625	80	100	123	122	120	80	1	False
4	4	Charmander	Fire	NaN	309	39	52	43	60	50	65	1	False
...	...	...	...	...	...	...	...	...	...	...	...	...	...
795	719	Diancie	Rock	Fairy	600	50	100	150	100	150	50	6	True
796	719	DiancieMega Diancie	Rock	Fairy	700	50	160	110	160	110	110	6	True
797	720	HoopaHoopa Confined	Psychic	Ghost	600	80	110	60	150	130	70	6	True
798	720	HoopaHoopa Unbound	Psychic	Dark	680	80	160	60	170	130	80	6	True
799	721	Volcanion	Fire						0	90	70	6	True

How about first and last rows?

800 rows x 13 columns

Problem A

❖ Save csv

```
1 data.to_csv('modified.csv')
```

To csv format.

```
1 data.to_excel('modified.xlsx', index = False)
```

To xlsx format, remove index col.

```
1 data.to_csv('modified_tab.csv', sep='\t')
```

To CSV format with columns
seperated by 'tab'.

pandas.DataFrame.to_csv

```
DataFrame.to_csv(path_or_buf=None, *, sep=',', na_rep='',  
float_format=None, columns=None, header=True, index=True,  
index_label=None, mode='w', encoding=None, compression='infer',  
quoting=None, quotechar='"', lineterminator=None, chunksize=None,  
date_format=None, doublequote=True, escapechar=None, decimal='.',  
errors='strict', storage_options=None)
```

[\[source\]](#)

Write object to a comma-separated values (csv) file.

pandas.DataFrame.to_excel

```
DataFrame.to_excel(excel_writer, *, sheet_name='Sheet1', na_rep='',  
float_format=None, columns=None, header=True, index=True,  
index_label=None, startrow=0, startcol=0, engine=None, merge_cells=True,  
inf_rep='inf', freeze_panes=None, storage_options=None,  
engine_kwargs=None)
```

[\[source\]](#)

Write object to an Excel sheet.

Problem A

❖ Describe method

pandas.DataFrame.describe

`DataFrame.describe(percentiles=None, include=None, exclude=None)` [\[source\]](#)

Generate descriptive statistics.

Descriptive statistics include those that summarize the central tendency, dispersion and shape of a dataset's distribution, excluding `NaN` values.

Analyzes both numeric and object series, as well as `DataFrame` column sets of mixed data types. The output will vary depending on what is provided. Refer to the notes below for more detail.

Problem A

❖ Describe method

To get some general information of the data

```
1 data.describe()
```

	#	Total	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed	Generation
count	800.000000	800.00000	800.000000	800.000000	800.000000	800.000000	800.000000	800.000000	800.00000
mean	362.813750	435.10250	69.258750	79.001250	73.842500	72.820000	71.902500	68.277500	3.32375
std	208.343798	119.96304	25.534669	32.457366	31.183501	32.722294	27.828916	29.060474	1.66129
min	1.000000	180.00000	1.000000	5.000000	5.000000	10.000000	20.000000	5.000000	1.00000
25%	184.750000	330.00000	50.000000	55.000000	50.000000	49.750000	50.000000	45.000000	2.00000
50%	364.500000	450.00000	65.000000	75.000000	70.000000	65.000000	70.000000	65.000000	3.00000
75%	539.250000	515.00000	80.000000	100.000000	90.000000	95.000000	90.000000	90.000000	5.00000
max	721.000000	780.00000	255.000000	190.000000	230.000000	194.000000	230.000000	180.000000	6.00000

Problem A

❖ Head/tail method

```
1 data.head()
```

#		Name	Type 1	Type 2	Total	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed	Generation	Legendary
0	1	Bulbasaur	Grass	Poison	318	45	49	49	65	65	45	1	False
1	2	Ivysaur	Grass	Poison	405	60	62	63	80	80	60	1	False
2	3	Venusaur	Grass	Poison	525	80	82	83	100	100	80	1	False
3	3	VenusaurMega Venusaur	Grass	Poison	625	80	100	123	122	120	80	1	False
4	4	Charmander	Fire	NaN	309	39	52	43	60	50	65	1	False

Problem A

❖ Head/tail method

```
1 data.tail(8)
```

#		Name	Type 1	Type 2	Total	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed	Generation	Legendary
792	716	Xerneas	Fairy	NaN	680	126	131	95	131	98	99	6	True
793	717	Yveltal	Dark	Flying	680	126	131	95	131	98	99	6	True
794	718	Zygarde50% Forme	Dragon	Ground	600	108	100	121	81	95	95	6	True
795	719	Diancie	Rock	Fairy	600	50	100	150	100	150	50	6	True
796	719	DiancieMega Diancie	Rock	Fairy	700	50	160	110	160	110	110	6	True
797	720	HoopaHoopa Confined	Psychic	Ghost	600	80	110	60	150	130	70	6	True
798	720	HoopaHoopa Unbound	Psychic	Dark	680	80	160	60	170	130	80	6	True
799	721	Volcanion	Fire	Water	600	80	110	120	130	90	70	6	True

Problem A

❖ Get headers

```
1 print(data.columns)
```

```
Index(['#', 'Name', 'Type 1', 'Type 2', 'Total', 'HP', 'Attack', 'Defense',  
      'Sp. Atk', 'Sp. Def', 'Speed', 'Generation', 'Legendary'],  
      dtype='object')
```

pandas.DataFrame.columns

DataFrame.columns

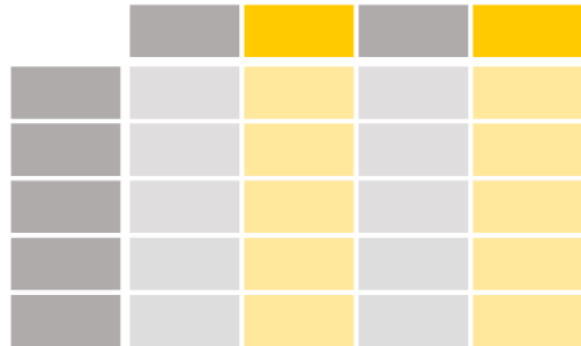
The column labels of the DataFrame.

Examples

```
>>> df = pd.DataFrame({'A': [1, 2], 'B': [3, 4]})  
>>> df  
   A  B  
0  1  3  
1  2  4  
>>> df.columns  
Index(['A', 'B'], dtype='object')
```

Problem A

❖ Get specific columns






```
1 print(data[['Name']].head())
2 print('\n')
3 print(data[['Name', 'Total', 'Attack']][:10:2])
```

	Name
0	Bulbasaur
1	Ivysaur
2	Venusaur
3	VenusaurMega Venusaur
4	Charmander

	Name	Total	Attack
0	Bulbasaur	318	49
2	Venusaur	525	82
4	Charmander	309	52
6	Charizard	534	84
8	CharizardMega Charizard Y	634	104

Problem A

❖ Change specific columns: Remove

To remove a specific column, use **drop()** method.

pandas.DataFrame.drop

DataFrame.drop(*labels=None, *, axis=0, index=None, columns=None, level=None, inplace=False, errors='raise'*)

[\[source\]](#)

Drop specified labels from rows or columns.

Remove rows or columns by specifying label names and corresponding axis, or by directly specifying index or column names. When using a multi-index, labels on different levels can be removed by specifying the level. See the [user guide](#) for more information about the now unused levels.

```
1 data.head(3)
```

	#	Name	Type 1	Type 2	Total	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed	Generation	Legendary
0	1	Bulbasaur	Grass	Poison	318	45	49	49	65	65	45	1	False
1	2	Ivysaur	Grass	Poison	405	60	62	63	80	80	60	1	False
2	3	Venusaur	Grass	Poison	525	80	82	83	100	100	80	1	False

```
1 data = data.drop(columns=['Total'])  
2 data.head(3)
```

	#	Name	Type 1	Type 2	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed	Generation	Legendary
0	1	Bulbasaur	Grass	Poison	45	49	49	65	65	45	1	False
1	2	Ivysaur	Grass	Poison	60	62	63	80	80	60	1	False
2	3	Venusaur	Grass	Poison	80	82	83	100	100	80	1	False

Problem A

❖ Change specific columns: Create

```
1 data['Total'] = data['Attack'] + data['Defense'] + data['HP']  
2 data.head(3)
```

	#	Name	Type 1	Type 2	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed	Generation	Legendary	Total
0	1	Bulbasaur	Grass	Poison	45	49	49	65	65	45	1	False	143
1	2	Ivysaur	Grass	Poison	60	62	63	80	80	60	1	False	185
2	3	Venusaur	Grass	Poison	80	82	83	100	100	80	1	False	245

To add a specific column, just assign a new Series.

Problem A

❖ Arranging order

```
1 cols = list(data.columns.values)
2 print(cols)
```

```
['#', 'Name', 'Type 1', 'Type 2',  
'HP', 'Attack', 'Defense', 'Sp.  
Atk', 'Sp. Def', 'Speed',  
'Generation', 'Legendary',  
'Total']
```

```
1 data[['Name', 'Total', 'Attack', 'Speed']].head()
```

	Name	Total	Attack	Speed
0	Bulbasaur	143	49	45
1	Ivysaur	185	62	60
2	Venusaur	245	82	80
3	VenusaurMega Venusaur	303	100	80
4	Charmander	134	52	65

Rearrange or reorder columns in a Pandas DataFrame by name indexing.

Problem A

❖ Arranging order

Rearrange or reorder columns in a Pandas DataFrame by slicing and combining different segments of the column list.

```
1 cols = list(data.columns.values)
2 print(cols)
```

```
['#', 'Name', 'Type 1', 'Type 2',
'HP', 'Attack', 'Defense', 'Sp.
Atk', 'Sp. Def', 'Speed',
'Generation', 'Legendary',
'Total']
```

```
1 data[cols[0:4] + [cols[-1]] + cols[4:-1]].head(3)
```

	#	Name	Type 1	Type 2	Total	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed	Generation	Legendary
0	1	Bulbasaur	Grass	Poison	143	45	49	49	65	65	45	1	False
1	2	Ivysaur	Grass	Poison	185	60	62	63	80	80	60	1	False
2	3	Venusaur	Grass	Poison	245	80	82	83	100	100	80	1	False

Problem A

❖ Selecting rows: By index

```
1 data.iloc[[3]]
```

#	Name	Type 1	Type 2	Total	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed	Generation	Legendary
3	VenusaurMega Venusaur	Grass	Poison	625	80	100	123	122	120	80	1	False

```
1 data.iloc[[3,6]]
```

#	Name	Type 1	Type 2	Total	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed	Generation	Legendary
3	VenusaurMega Venusaur	Grass	Poison	625	80	100	123	122	120	80	1	False
6	Charizard	Fire	Flying	534	78	84	78	109	85	100	1	False

```
1 data.iloc[3:5]
```

#	Name	Type 1	Type 2	Total	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed	Generation	Legendary
3	VenusaurMega Venusaur	Grass	Poison	625	80	100	123	122	120	80	1	False
4	Charmander	Fire	NaN	309	39	52	43	60	50	65	1	False

pandas.DataFrame.iloc

property **DataFrame.iloc**

[\[source\]](#)

Purely integer-location based indexing for selection by position.

⚠️ *Deprecated since version 2.2.0:* Returning a tuple from a callable is deprecated.

`.iloc[]` is primarily integer position based (from `0` to `length-1` of the axis), but may also be used with a boolean array.

Allowed inputs are:

- An integer, e.g. `5`.
- A list or array of integers, e.g. `[4, 3, 0]`.
- A slice object with ints, e.g. `1:7`.
- A boolean array.
- A `callable` function with one argument (the calling Series or DataFrame) and that returns valid output for indexing (one of the above). This is useful in method chains, when you don't have a reference to the calling object, but would like to base your selection on some value.
- A tuple of row and column indexes. The tuple elements consist of one of the above inputs, e.g. `(0, 1)`.

`.iloc` will raise `IndexError` if a requested indexer is out-of-bounds, except *slice* indexers which allow out-of-bounds indexing (this conforms with python/numpy *slice* semantics).

See more at [Selection by Position](#).

Problem A

❖ Selecting rows: By condition

```
1 data.loc[data['Legendary'] == True].head(2)
```

	#	Name	Type 1	Type 2	Total	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed	Generation	Legendary
156	144	Articuno	Ice	Flying	580	90	85	100	95	125	85	1	True
157	145	Zapdos	Electric	Flying	580	90	90	85	125	90	100	1	True

pandas.DataFrame.loc

property DataFrame.**loc**

[\[source\]](#)

Access a group of rows and columns by label(s) or a boolean array.

`.loc[]` is primarily label based, but may also be used with a boolean array.

Allowed inputs are:

- A single label, e.g. `5` or `'a'`, (note that `5` is interpreted as a *label* of the index, and **never** as an integer position along the index).
- A list or array of labels, e.g. `['a', 'b', 'c']`.
- A slice object with labels, e.g. `'a':'f'`.

⚠ Warning

Note that contrary to usual python slices, **both** the start and the stop are included

- A boolean array of the same length as the axis being sliced, e.g. `[True, False, True]`.
- An alignable boolean Series. The index of the key will be aligned before masking.
- An alignable Index. The Index of the returned selection will be the input.
- A `callable` function with one argument (the calling Series or DataFrame) and that returns valid output for indexing (one of the above)

See more at [Selection by Label](#).

Problem A

❖ Selecting rows: By condition

```
1 data.head()
```

#		Name	Type 1	Type 2	Total	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed	Generation	Legendary
0	1	Bulbasaur	Grass	Poison	318	45	49	49	65	65	45	1	False
1	2	Ivysaur	Grass	Poison	405	60	62	63	80	80	60	1	False
2	3	Venusaur	Grass	Poison	525	80	82	83	100	100	80	1	False
3	3	VenusaurMega Venusaur	Grass	Poison	625	80	100	123	122	120	80	1	False
4	4	Charmander	Fire	NaN	309	39	52	43	60	50	65	1	False

```
1 data.iloc[3,4]
```

625

```
1 data.loc[data['Attack'] == 49, ['Name']]
```

Name	
0	Bulbasaur
166	Chikorita
506	Finneon

Problem A

❖ Filtering: By .loc

```
1 data.loc[(data['Type 1'] == 'Grass') & (data['Type 2'] == 'Poison')].head(3)
```

	#	Name	Type 1	Type 2	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed	Generation	Legendary	Total
0	1	Bulbasaur	Grass	Poison	45	49	49	65	65	45	1	False	143
1	2	Ivysaur	Grass	Poison	60	62	63	80	80	60	1	False	185
2	3	Venusaur	Grass	Poison	80	82	83	100	100	80	1	False	245

```
1 data.loc[(data['Type 1'] == 'Grass') | (data['Type 2'] == 'Poison')][0::10].loc[data['HP'] > 50].head(3)
```

	#	Name	Type 1	Type 2	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed	Generation	Legendary	Total
50	45	Vileplume	Grass	Poison	75	80	85	110	90	50	1	False	240
101	94	Gengar	Ghost	Poison	60	65	60	130	75	110	1	False	185
197	182	Bellossom	Grass	NaN	75	80	95	90	100	50	2	False	250

Problem A

❖ Filtering: By regex

```
1 import re
```

```
1 data.loc[data['Name'].str.contains('Mega')].head(3)
```

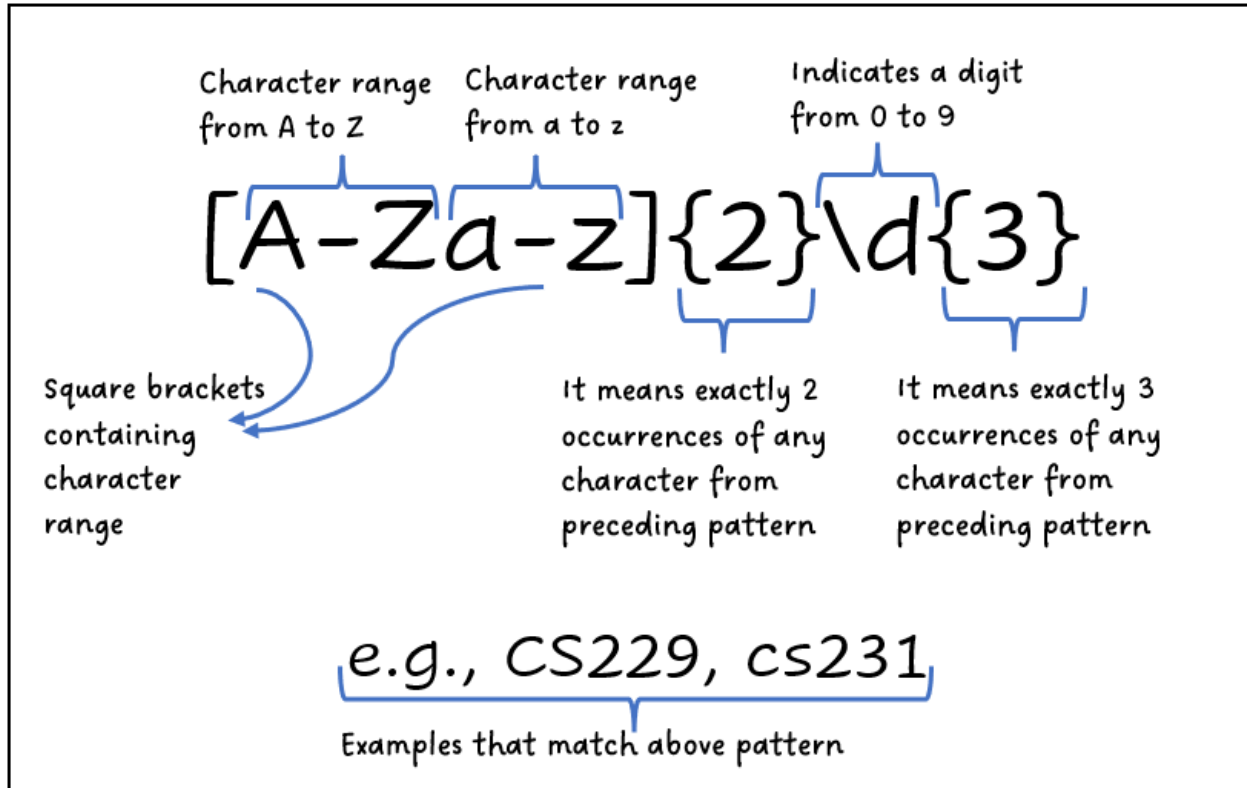
#		Name	Type 1	Type 2	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed	Generation	Legendary	Total
3	3	VenusaurMega Venusaur	Grass	Poison	80	100	123	122	120	80	1	False	303
7	6	CharizardMega Charizard X	Fire	Dragon	78	130	111	130	85	100	1	False	319
8	6	CharizardMega Charizard Y	Fire	Flying	78	104	78	159	115	100	1	False	260

```
1 data.loc[data['Type 1'].str.contains('Fire|grass', regex=True)].head(3)
```

#		Name	Type 1	Type 2	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed	Generation	Legendary	Total
4	4	Charmander	Fire	NaN	39	52	43	60	50	65	1	False	134
5	5	Charmeleon	Fire	NaN	58	64	58	80	65	80	1	False	180
6	6	Charizard	Fire	Flying	78	84	78	109	85	100	1	False	240

Problem A

❖ Filtering: By regex



A regular expression (shortened as regex or regexp) is a sequence of characters that specifies a match pattern in text. Usually such patterns are used by string-searching algorithms for "find" or "find and replace" operations on strings, or for input validation.

Problem A

❖ Filtering: By regex

REGEX basic syntaxs

REGEX SYNTAX	MEANING	EXAMPLE	MATCHES	DOES NOT MATCH
.	Any single character	go.gle	google, goggle	gogle
[abc]	Any of these character	analy[zs]e	analyse, analyze	analyxe
[a-z]	Any character in this range	demo[2-4]	demo2, demo3	demo1, demo5
[^abc]	None of these characters	analy[^zs]e	analyxe	analyse, analyze
[^a-z]	Not a character in this range	demo[^2-4]	demo1, demo5	demo2, demo3
	Or	demo example	demo, demos, example	test
^	Starts with	^demo	demos, demonstration	my demo
\$	Ends with	demo\$	my demo	demonstration
?	Zero or one times (greedy)	demos?123	demo123, demos123	demoA123
??	Zero or one times (lazy)			
*	Zero or more times (greedy)	goo*gle	gogle, goooogle	goggle
*?	Zero or more times (lazy)			
+	One or more times (greedy)	goo+gle	google, goooogle	gogle, goggle
+?	One or more times (lazy)			
{n}	n times exactly	w{3}	www	w, ww
{n,m}	from n to m times	a{4, 7}	aaaa, aaaaa, aaaaaa, aaaaaaa	aaaaaaaa, aaa, a
{n,}	at least n times	go{2,}gle	google, gooogle, goooogle	ggle, gogle
()	Group	^(demo example)[0-9]+	demo1, example4	demoexample2
(?:)	Passive group (Useful for filters)			
\	Escape	AU\\$10	AU\$10, AU\$100	AU10, 10
\s	White space			
\S	Non-white space			
\d	Digit character			
\D	Non-digit character			
\w	Word			
\W	Non-word (e.g. punctuation, spaces)			

Problem A

❖ Filtering: By regex

```
1 data.loc[data['Name'].str.contains('pi[a-z]*', flags=re.I, regex=True)].head(3)
```

	#	Name	Type 1	Type 2	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed	Generation	Legendary	Total
13	10	Caterpie	Bug	NaN	45	30	35	20	20	45	1	False	110
20	16	Pidgey	Normal	Flying	40	45	40	35	35	56	1	False	125
21	17	Pidgeotto	Normal	Flying	63	60	55	50	50	71	1	False	178

```
1 data.loc[data['Name'].str.contains('^pi[a-z]*', flags=re.I, regex=True)].head(3)
```

	#	Name	Type 1	Type 2	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed	Generation	Legendary	Total
20	16	Pidgey	Normal	Flying	40	45	40	35	35	56	1	False	125
21	17	Pidgeotto	Normal	Flying	63	60	55	50	50	71	1	False	178
22	18	Pidgeot	Normal	Flying	83	80	75	70	70	101	1	False	238

```
1 data.loc[data['Name'].str.contains('^pi.*', flags=re.I, regex=True)].head(3)
```

	#	Name	Type 1	Type 2	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed	Generation	Legendary	Total
20	16	Pidgey	Normal	Flying	40	45	40	35	35	56	1	False	125
21	17	Pidgeotto	Normal	Flying	63	60	55	50	50	71	1	False	178
22	18	Pidgeot	Normal	Flying	83	80	75	70	70	101	1	False	238

Problem A

❖ Sorting

```
1 data.sort_values('Type 1').head(3)
```

	#	Name	Type 1	Type 2	Total	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed	Generation	Legendary
600	540	Sewaddle	Bug	Grass	310	45	53	70	40	60	42	5	False
136	127	Pinsir	Bug	NaN	500	65	125	100	55	70	85	1	False
457	412	Burmy	Bug	NaN	224	40	29	45	29	45	36	4	False

```
1 data.sort_values('Type 1', ascending=False).head(3)
```

	#	Name	Type 1	Type 2	Total	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed	Generation	Legendary
371	339	Barboach	Water	Ground	288	50	48	43	46	41	60	3	False
97	90	Shellder	Water	NaN	305	30	65	100	45	25	40	1	False
240	222	Corsola	Water	Rock	380	55	55	85	65	85	35	2	False

```
1 data.sort_values(['Type 1', 'HP'], ascending=[1, 0]).head(3)
```

	#	Name	Type 1	Type 2	Total	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed	Generation	Legendary
520	469	Yanmega	Bug	Flying	515	86	76	86	116	56	95	4	False
698	637	Volcarona	Bug	Fire	550	85	60	65	135	105	100	5	False
231	214	Heracross	Bug	Fighting	500	80	125	75	40	95	85	2	False

Problem A

❖ Reset index

```
1 new_data = data.loc[(data['Type 1'] == 'Grass') | (data['Type 2'] == 'Poison')][0::10].loc[data['HP'] > 50].head(3)
2 new_data
```

	#	Name	Type 1	Type 2	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed	Generation	Legendary	Total
50	45	Vileplume	Grass	Poison	75	80	85	110	90	50	1	False	240
101	94	Gengar	Ghost	Poison	60	65	60	130	75	110	1	False	185
197	182	Bellossom	Grass	NaN	75	80	95	90	100	50	2	False	250

```
1 new_data = new_data.reset_index()
2 new_data
```

	index	#	Name	Type 1	Type 2	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed	Generation	Legendary	Total
0	50	45	Vileplume	Grass	Poison	75	80	85	110	90	50	1	False	240
1	101	94	Gengar	Ghost	Poison	60	65	60	130	75	110	1	False	185
2	197	182	Bellossom	Grass	NaN	75	80	95	90	100	50	2	False	250

Problem A

❖ Reset index

```
1 new_data = new_data.reset_index(drop=True)
2 new_data
```

	#	Name	Type 1	Type 2	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed	Generation	Legendary	Total
0	45	Vileplume	Grass	Poison	75	80	85	110	90	50	1	False	240
1	94	Gengar	Ghost	Poison	60	65	60	130	75	110	1	False	185
2	182	Bellossom	Grass	NaN	75	80	95	90	100	50	2	False	250

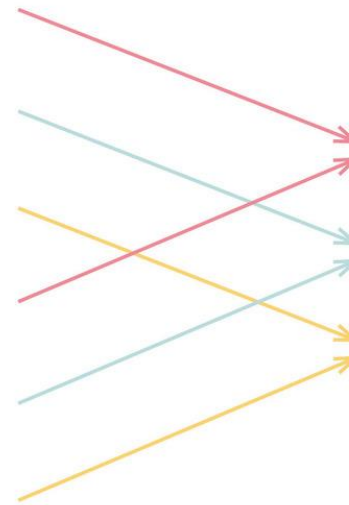
```
1 new_data.reset_index(drop=True, inplace = True)
2 new_data
```

	#	Name	Type 1	Type 2	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed	Generation	Legendary	Total
0	45	Vileplume	Grass	Poison	75	80	85	110	90	50	1	False	240
1	94	Gengar	Ghost	Poison	60	65	60	130	75	110	1	False	185
2	182	Bellossom	Grass	NaN	75	80	95	90	100	50	2	False	250

Problem A

❖ Groupby Operations

title	genre	price
book 1	adventure	11.90
book 2	fantasy	8.49
book 3	romance	9.99
book 4	adventure	9.99
book 5	fantasy	7.99
book 6	romance	5.88



genre	avg_price
adventure	$(11.90 + 9.99)/2$ 10.945
fantasy	$(8.49 + 7.99)/2$ 8.24
romance	$(9.99 + 5.88)/2$ 7.935

Groupby Operation

Problem A

❖ Groupby

```
1 data.groupby(['Type 1']).mean().sort_values('Defense', ascending=False).head(10)
```

<ipython-input-53-90d84ee3bc80>:1: FutureWarning: The default value of numeric_only in DataFrameGroupBy.mean is
data.groupby(['Type 1']).mean().sort_values('Defense', ascending=False).head(10)

	#	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed	Generation	Legendary	Total
Type 1										
Steel	442.851852	65.222222	92.703704	126.370370	67.518519	80.629630	55.259259	3.851852	0.148148	284.296296
Rock	392.727273	65.363636	92.863636	100.795455	63.340909	75.477273	55.909091	3.454545	0.090909	259.022727
Dragon	474.375000	83.312500	112.125000	86.375000	96.843750	88.843750	83.031250	3.875000	0.375000	281.812500
Ground	356.281250	73.781250	95.750000	84.843750	56.468750	62.750000	63.906250	3.156250	0.125000	254.375000
Ghost	486.500000	64.437500	73.781250	81.187500	79.343750	76.468750	64.343750	4.187500	0.062500	219.406250
Water	303.089286	72.062500	74.151786	72.946429	74.812500	70.517857	65.964286	2.857143	0.035714	219.160714
Ice	423.541667	72.000000	72.750000	71.416667	77.541667	76.291667	63.458333	3.541667	0.083333	216.166667
Grass	344.871429	67.271429	73.214286	70.800000	77.500000	70.428571	61.928571	3.357143	0.042857	211.285714
Bug	334.492754	56.884058	70.971014	70.724638	53.869565	64.797101	61.681159	3.217391	0.000000	198.579710
Dark	461.354839	66.806452	88.387097	70.225806	74.645161	69.516129	76.161290	4.032258	0.064516	225.419355

Problem A

❖ Grouper

```
1 data.groupby(['Type 1', 'Type 2']).count()
```

		#	Name	HP	Attack	Defense	Sp. Atk	Sp. Def	Speed	Generation	Legendary	Total
Type 1	Type 2											
Bug	Electric	2	2	2	2	2	2	2	2	2	2	2
	Fighting	2	2	2	2	2	2	2	2	2	2	2
	Fire	2	2	2	2	2	2	2	2	2	2	2
	Flying	14	14	14	14	14	14	14	14	14	14	14
	Ghost	1	1	1	1	1	1	1	1	1	1	1
...	...	...	...	...	...	...	...	...	...	...	...	...
Water	Ice	3	3	3	3	3	3	3	3	3	3	3
	Poison	3	3	3	3	3	3	3	3	3	3	3
	Psychic	5	5	5	5	5	5	5	5	5	5	5
	Rock	4	4	4	4	4	4	4	4	4	4	4
	Steel	1	1	1	1	1	1	1	1	1	1	1

136 rows x 11 columns

Problem A

❖ Groupby

```
1 df = data.groupby(['Type 1'])['Attack', 'Defense'].apply(lambda x: np.sum(x)/len(x))  
2 df.head(3)
```

```
<ipython-input-77-dcc0a50d3102>:1: FutureWarning: Indexing with multiple keys (implicitly  
df = data.groupby(['Type 1'])['Attack', 'Defense'].apply(lambda x: np.sum(x)/len(x))
```

Attack Defense

Type 1

Bug	70.971014	70.724638
Dark	88.387097	70.225806
Dragon	112.125000	86.375000

Problem A

❖ Groupby

```
1 df = data.groupby(['Type 1']).agg({'Attack': 'mean',  
2 |                        'Defense': 'sum',  
3 |                        'Speed': lambda x: len(x)})  
4 df.head(3)
```

	Attack	Defense	Speed
Type 1			
Bug	70.971014	4880	69
Dark	88.387097	2177	31
Dragon	112.125000	2764	32

Problem B

Problem B

❖ Problem statement

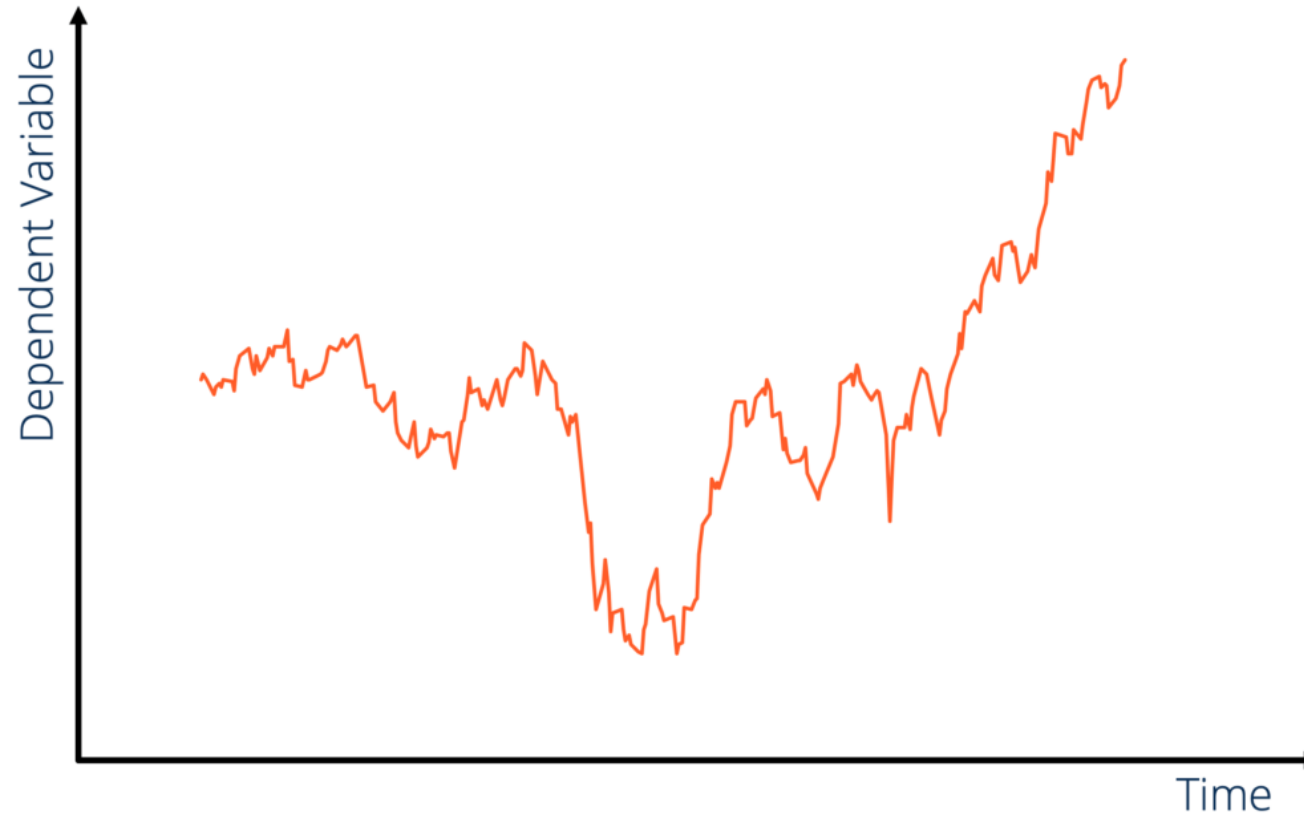
Description: Given [Daily minimum temperature dataset](#), use Pandas library to manipulate this dataset.

📅 Date	⚙	⚙ Daily minimum t...	⚙
	1981-01-01 1990-12-31	11	1%
		10	1%
		Other (3548)	97%
1/1/1981		20.7	
1/2/1981		17.9	
1/3/1981		18.8	
1/4/1981		14.6	
1/5/1981		15.8	
1/6/1981		15.8	
1/7/1981		15.8	
1/8/1981		17.4	
1/9/1981		21.8	

Problem B

❖ What is time-series data?

Time-Series Analysis



Problem B

❖ Problem statement

Description: Given [Daily minimum temperature dataset](#), use Pandas library to manipulate this dataset.

```
1 # Dataset B: https://drive.google.com/file/d/1fS6pGYj5FMZa8AsBHJ85jaiBJqWA4dqq/view?usp=sharing
2 !gdown --id 1fS6pGYj5FMZa8AsBHJ85jaiBJqWA4dqq
```

```
/usr/local/lib/python3.10/dist-packages/gdown/__main__.py:132: FutureWarning: Option `--id` was deprecated
  warnings.warn(
Downloading...
From: https://drive.google.com/uc?id=1fS6pGYj5FMZa8AsBHJ85jaiBJqWA4dqq
To: /content/timeseries_daily-minimum-temperatures.csv
100% 55.9k/55.9k [00:00<00:00, 58.1MB/s]
```

Problem B

❖ Read dataset

```
1 DATASET_PATH = './timeseries_daily-minimum-temperatures.csv'
2
3 # Read data from .csv file
4 ts_df = pd.read_csv(DATASET_PATH)
5
6 print(ts_df.shape)
7 print(ts_df.dtypes)
8 ts_df.head(3)
```

- Number of rows: 3650
- Number of columns: 2

(3650, 2)

Date object
Daily minimum temperatures object
dtype: object

	Date	Daily minimum temperatures
0	1/1/1981	20.7
1	1/2/1981	17.9
2	1/3/1981	18.8

Problem B

❖ Change index

Current index

	Date	Daily minimum temperatures
0	1/1/1981	20.7
1	1/2/1981	17.9
2	1/3/1981	18.8

* Since **Date** is also a unique value:

```
1 ts_df['Date'] = pd.to_datetime(ts_df['Date'])
2 ts_df = ts_df.set_index('Date')
3 ts_df.head(3)
```

Daily minimum temperatures

Date	
1981-01-01	20.7
1981-01-02	17.9
1981-01-03	18.8

New index

Problem B

❖ Add weekday, month, year

Daily minimum temperatures

Date	
1981-01-01	20.7
1981-01-02	17.9
1981-01-03	18.8

Date format: yy/mm/dd

pandas.DatetimeIndex

```
class pandas.DatetimeIndex(data=None, freq=_NoDefault.no_default,
tz=_NoDefault.no_default, normalize=_NoDefault.no_default,
closed=_NoDefault.no_default, ambiguous='raise', dayfirst=False,
yearfirst=False, dtype=None, copy=False, name=None)
```

[source]

Immutable ndarray-like of datetime64 data.

Represented internally as int64, and which can be boxed to Timestamp objects that are subclasses of datetime and carry metadata.

```
1 ts_df = pd.read_csv(DATASET_PATH, index_col=0, parse_dates=True)
2
3 ts_df['Year'] = ts_df.index.year
4 ts_df['Month'] = ts_df.index.month
5 ts_df['Weekday Name'] = ts_df.index.day_name()
6
7 ts_df.sample(5, random_state=0)
```

Daily minimum temperatures		Year	Month	Weekday Name
Date				
1990-08-20	9.7	1990	8	Monday
1982-07-24	6.6	1982	7	Saturday
1983-09-19	12	1983	9	Monday
1986-09-28	6.9	1986	9	Sunday
1988-01-12	18.3	1988	1	Tuesday



Problem B

❖ Add weekday, month, year

```
1 ts_df.dtypes
```

```
0
Daily minimum temperatures    object
Year                          int32
Month                         int32
Weekday Name                  object
```

dtype: object

Numerical values are being represented as object (string).

```
1 ts_df['Daily minimum temperatures'] = pd.to_numeric(
2 |     ts_df['Daily minimum temperatures'], errors='coerce')
3 ts_df.dtypes
```

```
0
Daily minimum temperatures    float64
Year                          int32
Month                         int32
Weekday Name                  object
```

Convert object to numerical data (float64).

dtype: object

pandas.to_numeric

```
pandas.to_numeric(arg, errors='raise', downcast=None,
dtype_backend=_NoDefault.no_default)
```

Convert argument to a numeric type.

[\[source\]](#)

Problem B

❖ Time-based indexing

Daily minimum temperatures

Date	
1981-01-01	20.7
1981-01-02	17.9
1981-01-03	18.8

DateTimeIndex

pandas.DataFrame.loc

property **DataFrame.loc**

Access a group of rows and columns by label(s) or a boolean array.

`.loc[]` is primarily label based, but may also be used with a boolean array.

[\[source\]](#)

```
1 ts_df.loc['1990-01-01':'1991-01-01']
```

Daily minimum temperatures	Year	Month	Weekday	Name
Date				
1990-01-01	14.8	1990	1	Monday
1990-01-02	13.3	1990	1	Tuesday
1990-01-03	15.6	1990	1	Wednesday

```
1 ts_df.loc['1990-01']
```

Daily minimum temperatures	Year	Month	Weekday	Name
Date				
1990-01-01	14.8	1990	1	Monday
1990-01-02	13.3	1990	1	Tuesday
1990-01-03	15.6	1990	1	Wednesday

Problem B

❖ Visualize time-series data

```
1 sns.set(rc={'figure.figsize':(9, 4)})
2 col_to_plot = 'Daily minimum temperatures'
3 ts_df[col_to_plot].plot(linewidth=0.8)
4 plt.ylabel(col_to_plot)
5 plt.show()
```

Time-series data plot

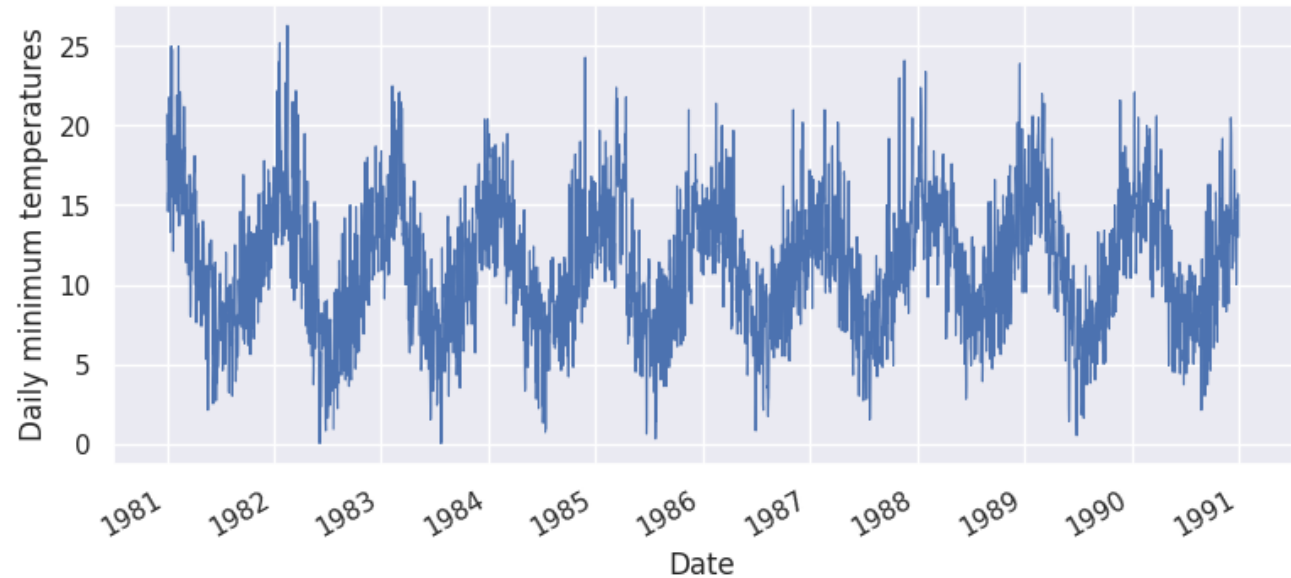
pandas.DataFrame.plot

DataFrame.plot(*args, **kwargs)

[\[source\]](#)

Make plots of Series or DataFrame.

Uses the backend specified by the option `plotting.backend`. By default, matplotlib is used.



Problem B

❖ Visualize time-series data

```
1 cols_plot = ['Daily minimum temperatures']
2 ts_df[cols_plot].plot(
3     marker='.',
4     alpha=0.5,
5     linestyle='None',
6     figsize=(10, 4),
7     subplots=True
8 )
9 plt.ylabel('Daily minimum temperatures')
10 plt.show()
```

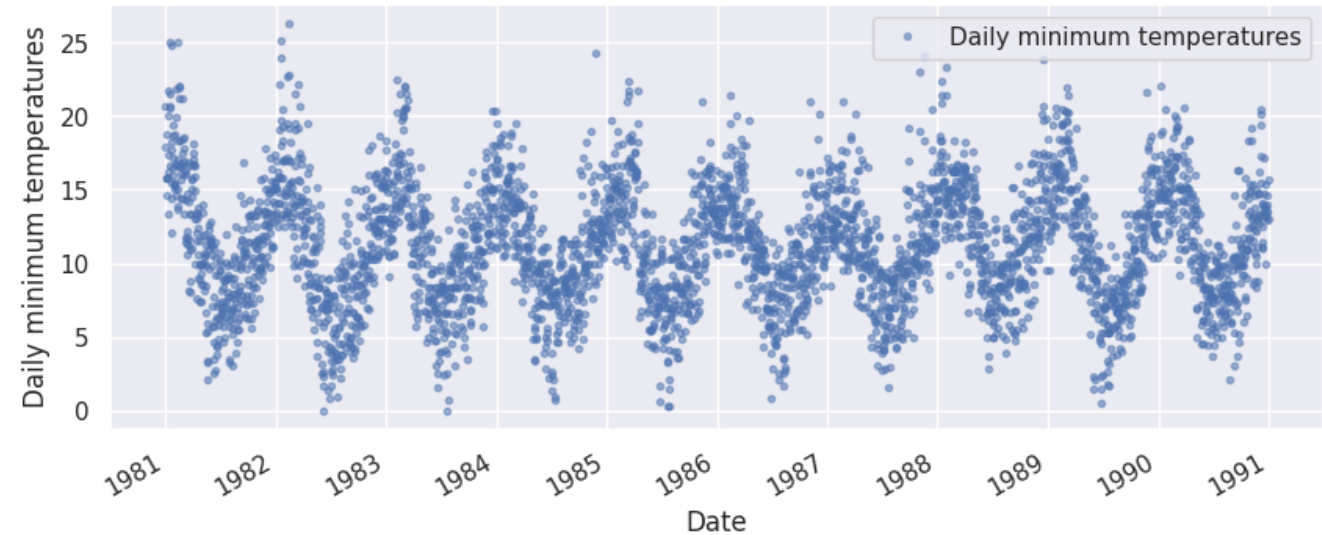
pandas.DataFrame.plot

DataFrame.plot(*args, **kwargs)

[\[source\]](#)

Make plots of Series or DataFrame.

Uses the backend specified by the option `plotting.backend`. By default, matplotlib is used.



Problem B

❖ Seasonality visualization



Seasonality is a characteristic of a time series in which the data experiences regular and predictable changes that recur every calendar year.

Problem B

❖ Seasonality visualization

```
1 COL_NAME = 'Daily minimum temperatures'
2 plt.figure(figsize=(10, 4))
3 sns.boxplot(data=ts_df, x='Month', y=COL_NAME)
4 plt.ylabel('Temperature')
5 plt.title(COL_NAME)
6 plt.show()
```

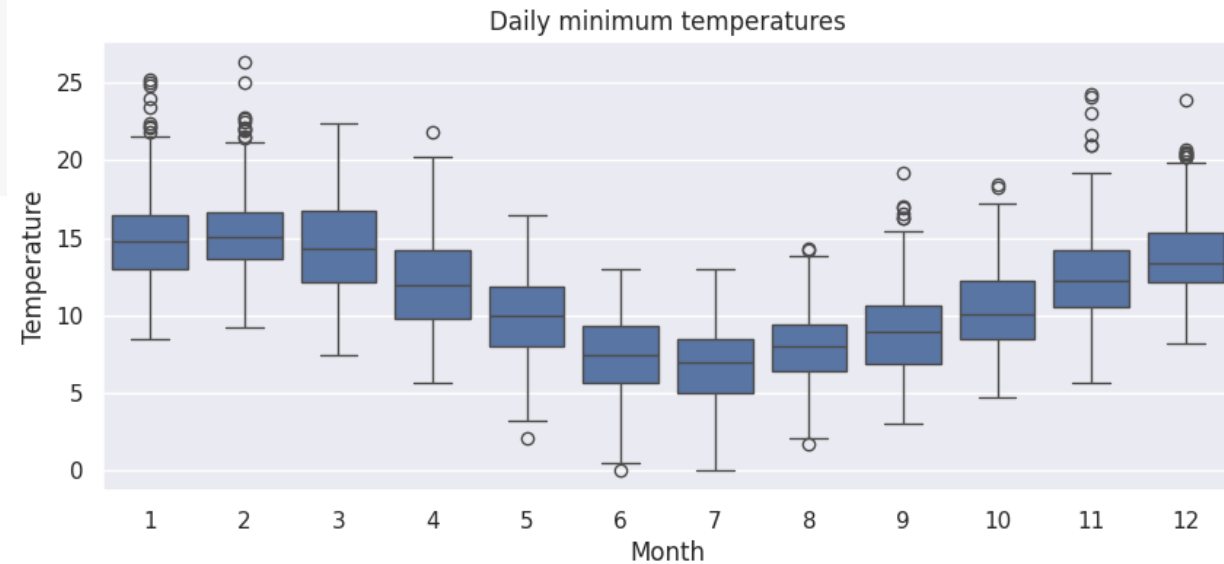
seaborn.boxplot

`seaborn.boxplot`(data=None, *, x=None, y=None, hue=None, order=None, hue_order=None, orient=None, color=None, palette=None, saturation=0.75, fill=True, dodge='auto', width=0.8, gap=0, whis=1.5, linecolor='auto', linewidth=None, fliersize=None, hue_norm=None, native_scale=False, log_scale=None, formatter=None, legend='auto', ax=None, **kwargs)

Draw a box plot to show distributions with respect to categories.

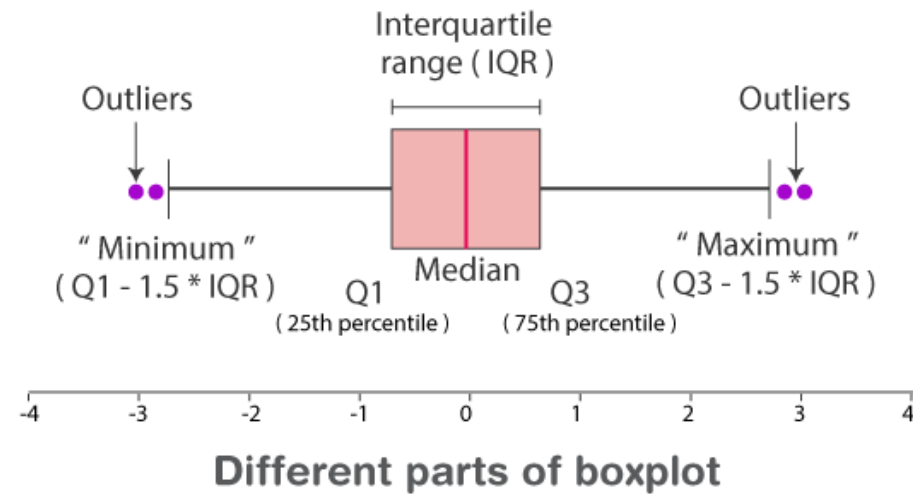
A box plot (or box-and-whisker plot) shows the distribution of quantitative data in a way that facilitates comparisons between variables or across levels of a categorical variable. The box shows the quartiles of the dataset while the whiskers extend to show the rest of the distribution, except for points that are determined to be “outliers” using a method that is a function of the inter-quartile range.

See the [tutorial](https://seaborn.pydata.org/generated/seaborn.boxplot.html) for more information.

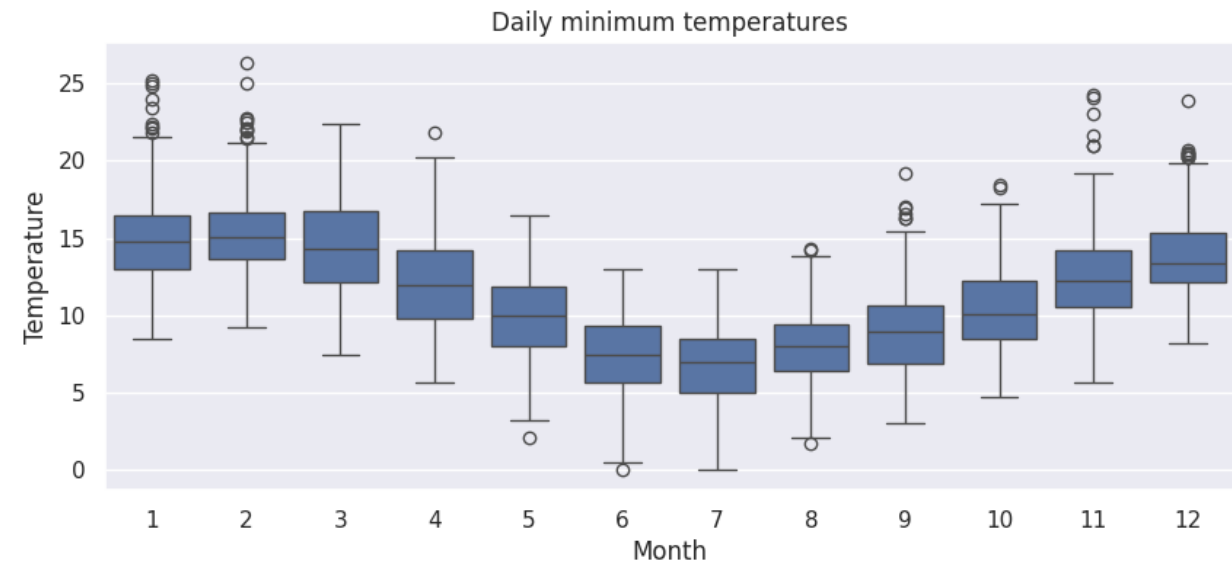


Problem B

❖ Seasonality visualization

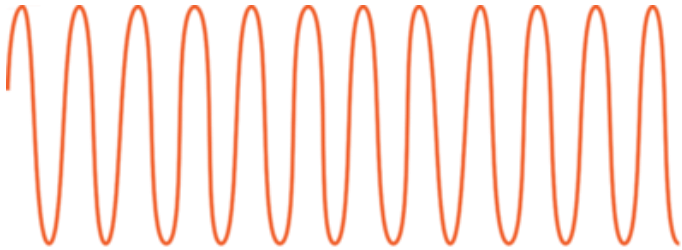


© Byjus.com



Problem B

❖ Frequencies



High
frequency

Day frequency

```
1 pd.date_range('1998-03-10', '1998-03-15', freq='D')
```

```
DatetimeIndex(['1998-03-10', '1998-03-11', '1998-03-12', '1998-03-13',  
              '1998-03-14', '1998-03-15'],  
              dtype='datetime64[ns]', freq='D')
```



Low
frequency

Hour frequency

```
1 pd.date_range('2004-09-20', periods=8, freq='H')
```

```
DatetimeIndex(['2004-09-20 00:00:00', '2004-09-20 01:00:00',  
              '2004-09-20 02:00:00', '2004-09-20 03:00:00',  
              '2004-09-20 04:00:00', '2004-09-20 05:00:00',  
              '2004-09-20 06:00:00', '2004-09-20 07:00:00'],  
              dtype='datetime64[ns]', freq='H')
```

Problem B

❖ Frequencies

```
1 # To select an arbitrary sequence of date/time values from a pandas time series,  
2 # we need to use a DatetimeIndex, rather than simply a list of date/time strings  
3 times_sample = pd.to_datetime(['1981-01-01', '1981-01-04', '1981-01-08'])  
4 # Select the specified dates and just the Consumption column  
5 consum_sample = ts_df.loc[times_sample, ['Daily minimum temperatures']].copy()  
6 consum_sample
```

Daily minimum temperatures		
1981-01-01	20.7	 
1981-01-04	14.6	
1981-01-08	17.4	

Assume we have a random 3 time series data point.

Problem B

❖ Frequencies

```
1 # Convert the data to daily frequency, without filling any missings
2 consum_freq = consum_sample.asfreq('D')
3 # Create a column with missings forward filled
4 consum_freq['Daily minimum temperatures - Forward Fill'] = consum_sample.asfreq('D', method='ffill')
5 consum_freq
```

	Daily minimum temperatures	Daily minimum temperatures - Forward Fill
1981-01-01	20.7	20.7
1981-01-02	NaN	20.7
1981-01-03	NaN	20.7
1981-01-04	14.6	14.6
1981-01-05	NaN	14.6
1981-01-06	NaN	14.6
1981-01-07	NaN	14.6
1981-01-08	17.4	17.4

	date	fruit	price
0	2021-01-01	apple	0.8
1	2021-01-02	apple	NaN
2	2021-01-03	apple	NaN
3	2021-01-04	apple	1.2
4	2021-01-01	mango	NaN
5	2021-01-02	mango	3.1
6	2021-01-03	mango	NaN
7	2021-01-04	mango	2.8

Diagram illustrating forward filling (ffill) for the 'price' column. Arrows labeled 'fill' point from the last non-NaN value in a group to the subsequent NaN values. For 'apple', the value 0.8 is filled forward to rows 1 and 2. For 'mango', the value 3.1 is filled forward to row 6.

	date	fruit	price
0	2021-01-01	apple	0.8
1	2021-01-02	apple	0.8
2	2021-01-03	apple	0.8
3	2021-01-04	apple	1.2
4	2021-01-01	mango	1.2
5	2021-01-02	mango	3.1
6	2021-01-03	mango	3.1
7	2021-01-04	mango	2.8

pandas.DataFrame.asfreq

`DataFrame.asfreq(freq, method=None, how=None, normalize=False, fill_value=None)`

[source]

Convert time series to specified frequency.

Returns the original data conformed to a new index with the specified frequency.

Filling additional days with **forward filling (ffill)**: fill with the last real value.

Problem B

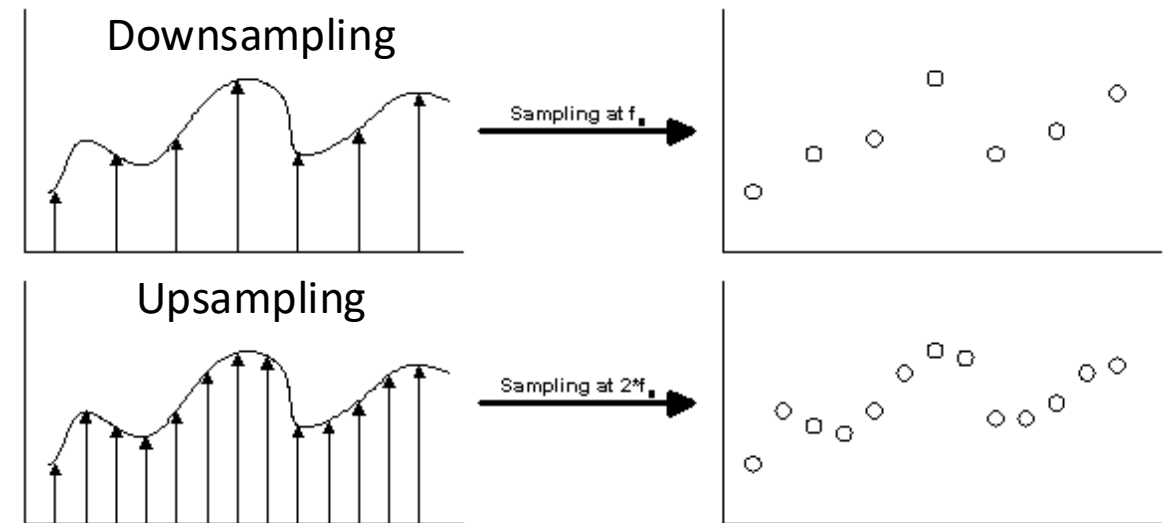
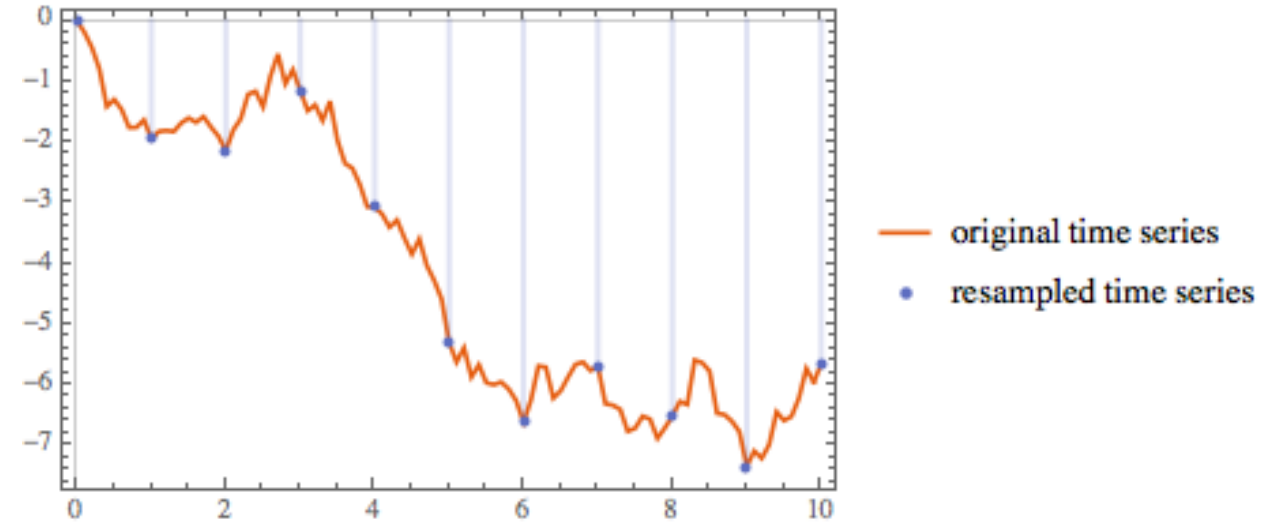
❖ Resampling

15/03	16/03	17/03	18/03	19/03	20/03	21/03
1	3	1	2	4	1	2

Time series by 1 day

15/03	17/03	19/03	21/03	23/03	25/03	27/03
2	1.5	2.5	...	...	...	...

Time series by 2 days



Problem B

❖ Resampling

```
1 data_columns = ['Daily minimum temperatures']
2 ts_weekly_mean = ts_df[data_columns].resample('W').mean()
3 ts_weekly_mean.head(3)
```

Downsampling to week frequency

Daily minimum temperatures



Date	
1981-01-04	18.000000
1981-01-11	17.542857
1981-01-18	20.371429

pandas.DataFrame.resample

DataFrame.resample(*rule*, *axis*=_NoDefault.no_default, *closed*=None, *label*=None, *convention*=_NoDefault.no_default, *kind*=_NoDefault.no_default, *on*=None, *level*=None, *origin*='start_day', *offset*=None, *group_keys*=False)

Resample time-series data.

[\[source\]](#)

Convenience method for frequency conversion and resampling of time series. The object must have a datetime-like index (*DatetimeIndex*, *PeriodIndex*, or *TimedeltaIndex*), or the caller must pass the label of a datetime-like series/index to the `on` / `level` keyword parameter.

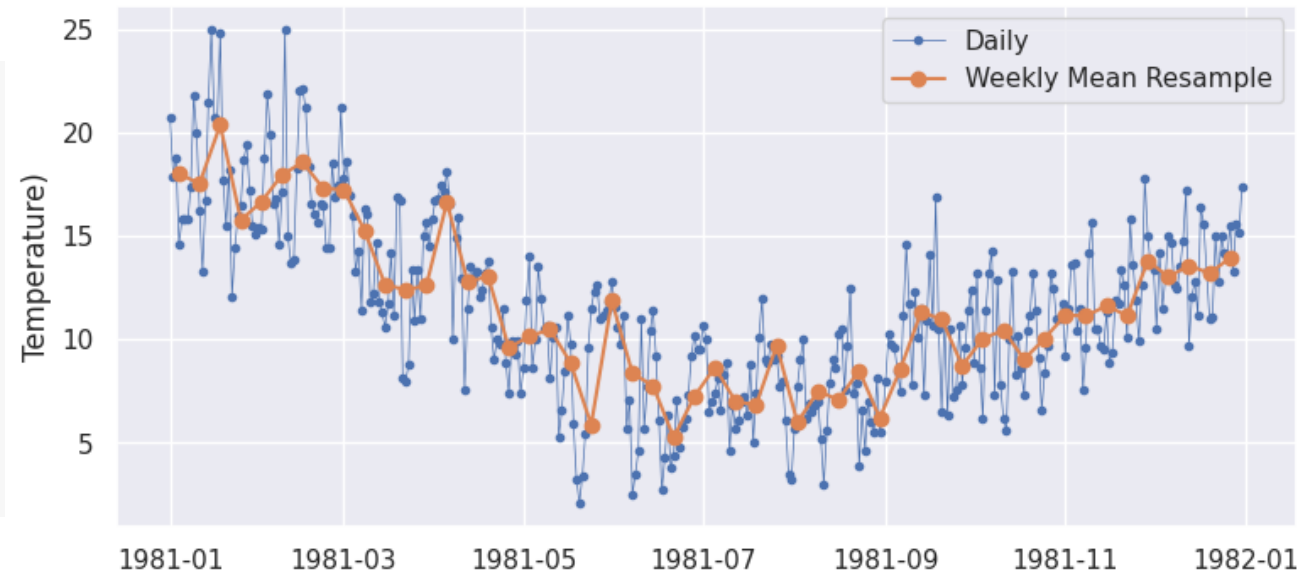
```
1 print(ts_df.shape[0])
2 print(ts_weekly_mean.shape[0])
```

3650
523

Problem B

❖ Resampling

```
1 # Start and end of the date range to extract
2 start, end = '1981-01', '1981-12'
3 # Plot daily and weekly resampled time series together
4 fig, ax = plt.subplots()
5 ax.plot(ts_df.loc[start:end, 'Daily minimum temperatures'],
6 marker='.', linestyle='-', linewidth=0.5, label='Daily')
7 ax.plot(ts_weekly_mean.loc[start:end, 'Daily minimum temperatures'],
8 marker='o', linestyle='-', label='Weekly Mean Resample')
9 ax.set_ylabel('Temperature')
10 ax.legend()
11 plt.show()
```



Problem B

❖ Resampling

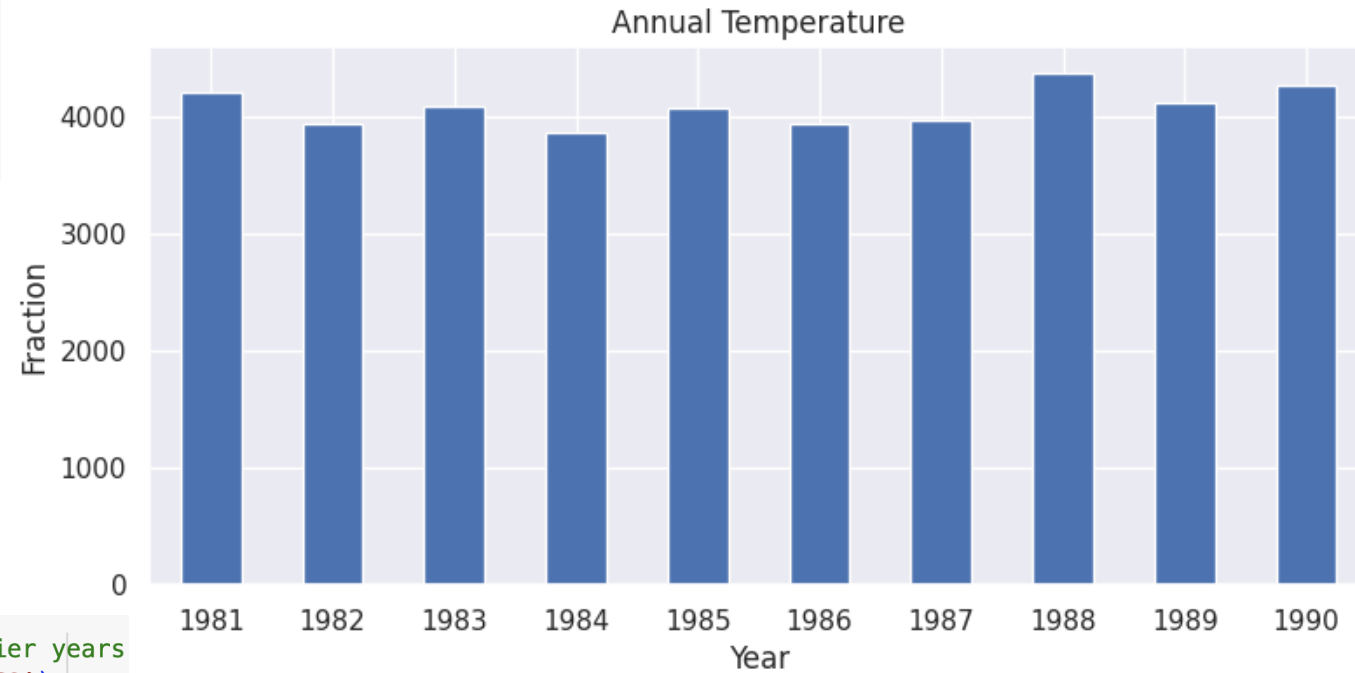
```
1 # Compute the annual sums, setting the value to NaN for any year which has
2 # fewer than 360 days of data
3 ts_annual_df = ts_df[data_columns].resample('A').sum(min_count=360)
4 # The default index of the resampled DataFrame is the last day of each year
5 # to the year component
6 ts_annual_df = ts_annual_df.set_index(ts_annual_df.index.year)
7 ts_annual_df.index.name = 'Year'
8 # Compute the ratio of Wind+Solar to Consumption
9 ts_annual_df.tail(3)
```

Daily minimum temperatures



Year

1988	4369.8
1989	4110.6
1990	4259.4



```
1 # Plot from 1981 onwards, because there is no solar production data in earlier years
2 ax = ts_annual_df.loc[1981:, 'Daily minimum temperatures'].plot.bar(color='C0')
3 ax.set_ylabel('Fraction')
4 ax.set_title('Annual Temperature')
5 plt.xticks(rotation=0)
6 plt.show()
```

QUIZ

❖ Summarization



Diagram illustrating the structure of a pandas DataFrame with axes and items.

Major Axis (vertical arrow pointing down):

Minor Axis (horizontal arrow pointing right):

Items (diagonal arrow pointing up-right):

Index	Name	Gender
0	A	M
5	E	M
9	I	F
10	J	M
11	K	F
12	L	F

In this lecture, we have discussed:

1. Introduction to Pandas library.
2. How to use pandas: from basic read and write csv to complex data manipulation.

Question

